

ECE/CS 757 Advanced Computer Architecture II

Spring 2025

GPUs

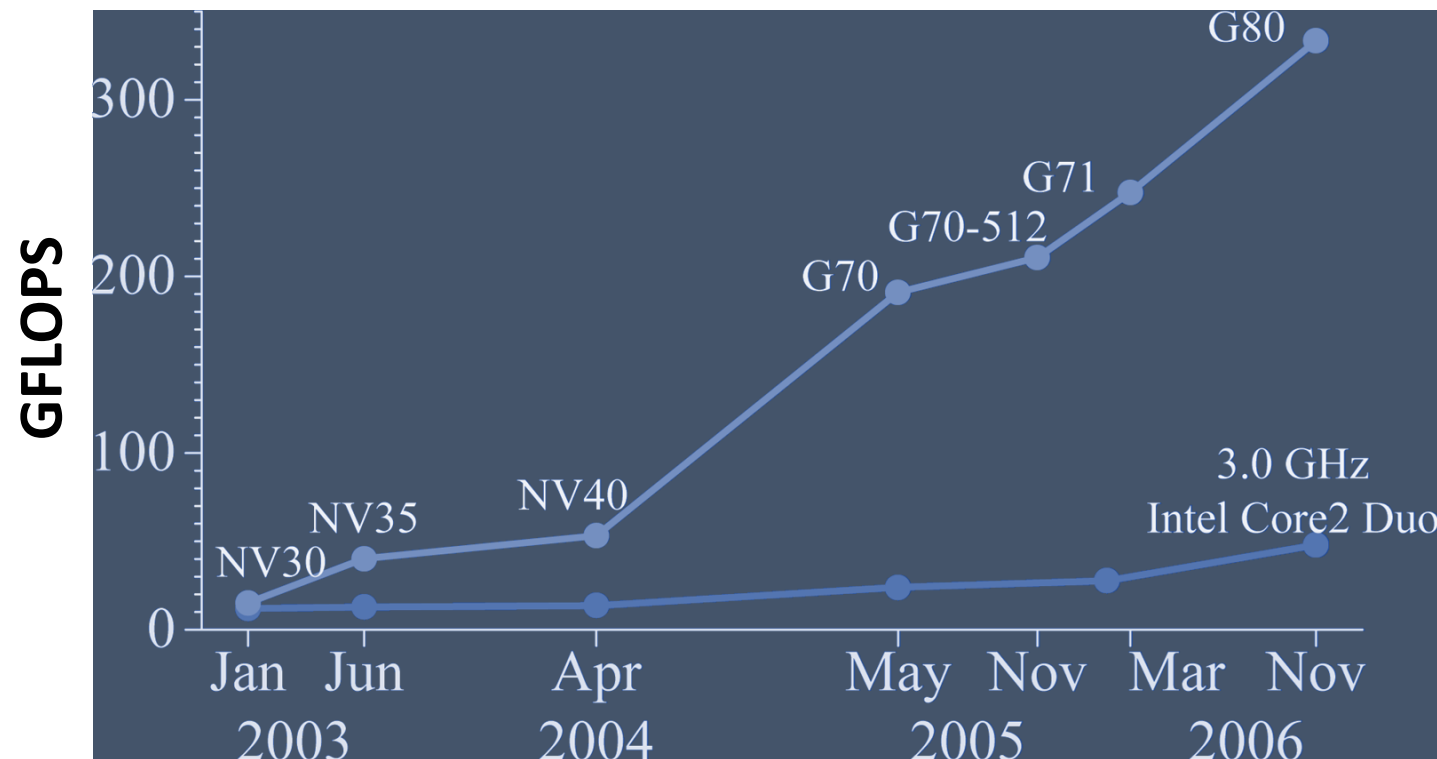
Instructor: Prof. Joshua San Miguel

Some slides adopted from Kayvon Fatahalian, Justin Hensley, Wen-mei Hwu, Natalie Enright Jerger, David Kirk, Mikko Lipasti and David Luebke

Graphics Processing Units

Killer app for massive data parallelism: graphics (e.g., 3D games)

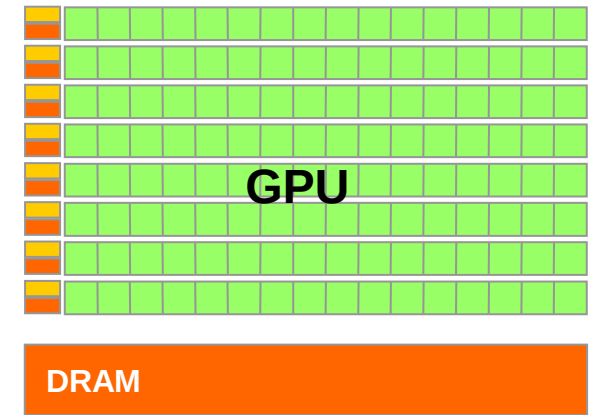
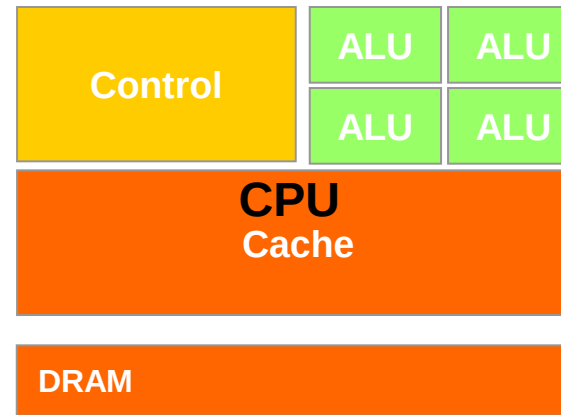
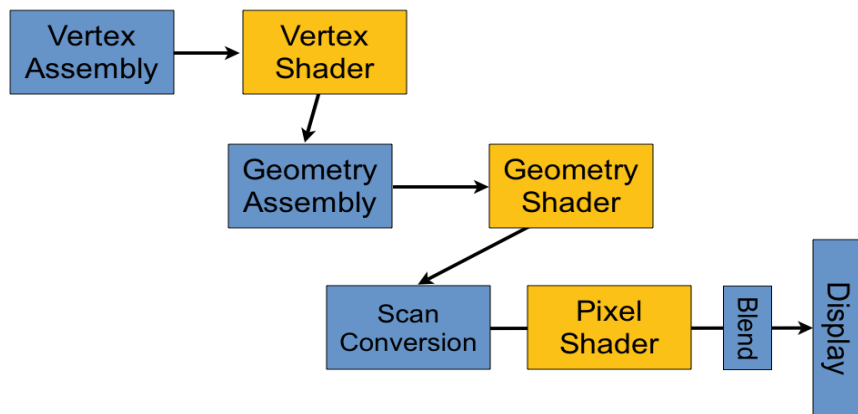
- Propelled by economic pressure of fast-growing video game industry
- GPU in every desktop, laptop, mobile device



Graphics Processing Units

GPUs

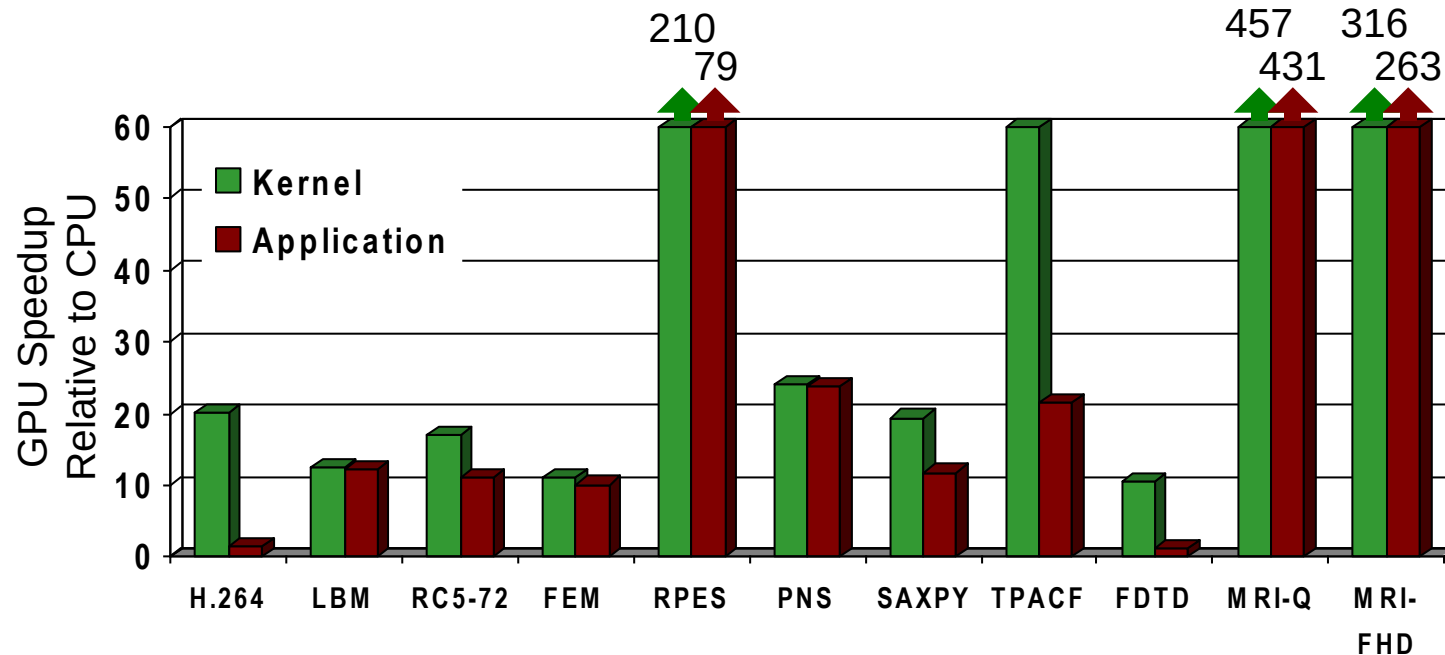
- Shader cores that exploit compute-intensive, massive data parallelism
- Devote more transistors to processing rather than caching and flow control
- Push for general-purpose use (GPGPUs) in high-performance computing



Graphics Processing Units

GPUs vs. CPUs

- 10x kernel speedup typical, as long as it can occupy enough parallel threads
- 25x-400x speedup if kernel's data requirements and control flow suit the GPU and the application is optimized



GeForce 8800 GTX vs. 2.2GHz Opteron 248

source: David Kirk/NVIDIA and Wen-mei W. Hwu, ECE498AL, UIUC, 2007

Graphics Processing Units

Target applications where:

- There are many, many threads
- Threads perform very similar operations
- Threads are compute-intensive
- Threads have simple control flow
- Threads are mostly independent

Graphics Processing Units

Target applications where:

- There are many, many threads
- Threads perform very similar operations
- Threads are compute-intensive
- Threads have simple control flow
- Threads are mostly independent

Design goals:

- Many cores; hide stall latencies via fine-grained context-switching of abundant threads
- Amortize complexities and overheads of fetch and decode
- Banked memories for maximum throughput (since latency less of an issue)
- Minimal (or no) support for speculation
- Minimal (or no) support for synchronization

Graphics Processing Units

Single program – multiple data (SPMD)

- Also viewed as single instruction – multiple threads (SIMT)
- Similar to SIMD and vector instructions, but coarser granularity of compute-level parallelism

Graphics Processing Units

Programming model

- CUDA or OpenCL, extensions to C
- Kernel = “shader task”, a snippet of scalar computation
- GPU executes kernel over many data elements; each invocation is independent

```
sampler mySamp;  
Texture2D<float3> myTex;  
float3 lightDir;  
float4 diffuseShader(float3 norm, float2 uv)  
{  
    float3 kd;  
    kd = myTex.Sample(mySamp, uv);  
    kd *= clamp( dot(lightDir, norm), 0.0, 1.0);  
    return float4(kd, 1.0);  
}
```



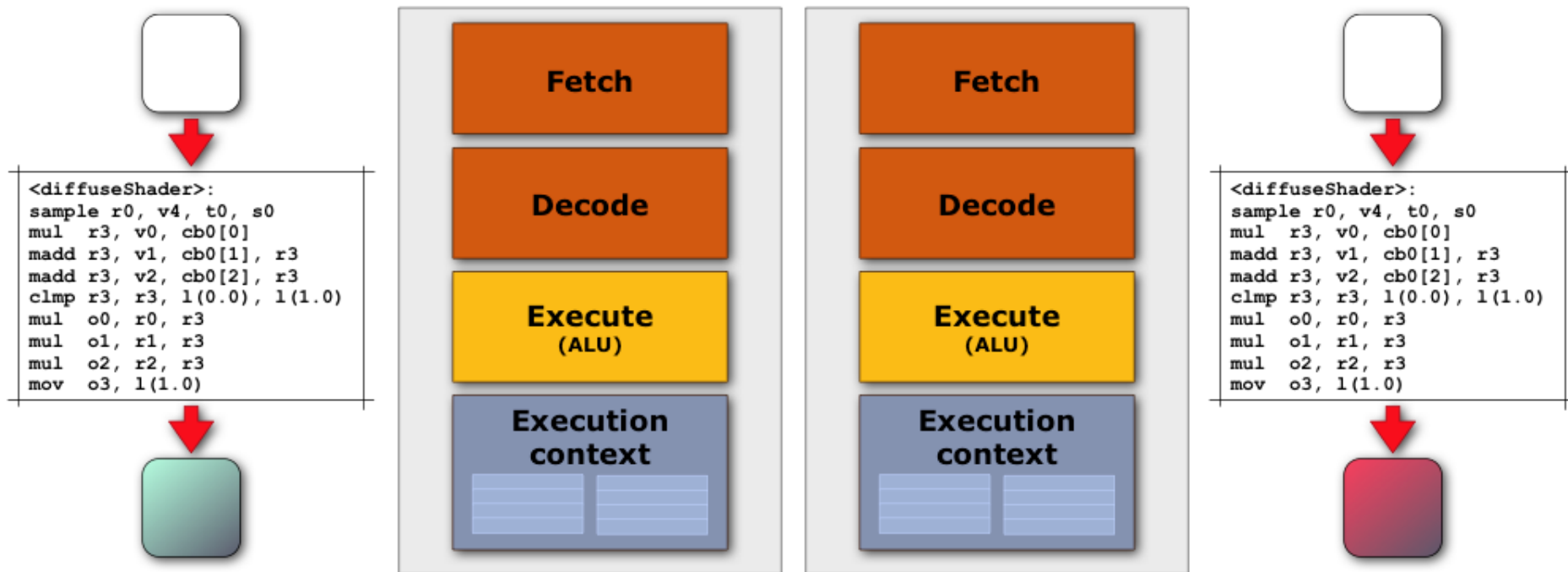
```
<diffuseShader>:  
sample r0, v4, t0, s0  
mul   r3, v0, cb0[0]  
madd  r3, v1, cb0[1], r3  
madd  r3, v2, cb0[2], r3  
clmp  r3, r3, 1(0.0), 1(1.0)  
mul   o0, r0, r3  
mul   o1, r1, r3  
mul   o2, r2, r3  
mov   o3, 1(1.0a)
```



Graphics Processing Units

Since all invocations of kernel are independent, run on many cores

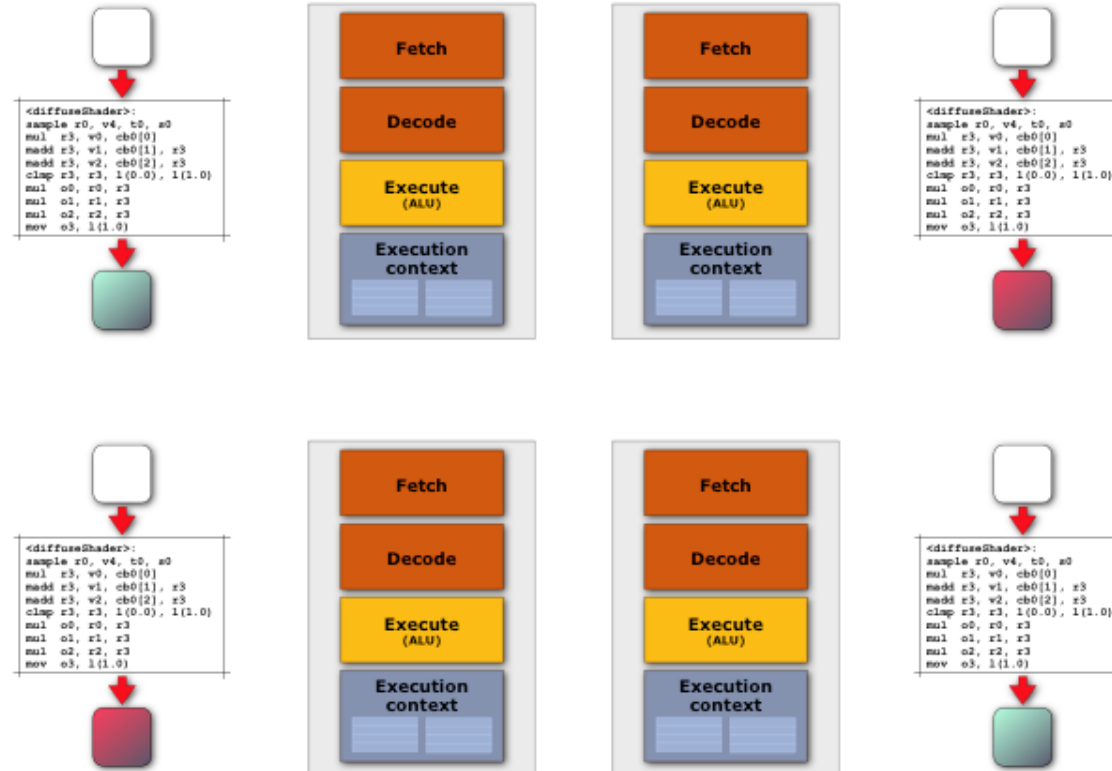
- Exploit the abundant parallelism



Graphics Processing Units

Since all invocations of kernel are independent, run on many cores

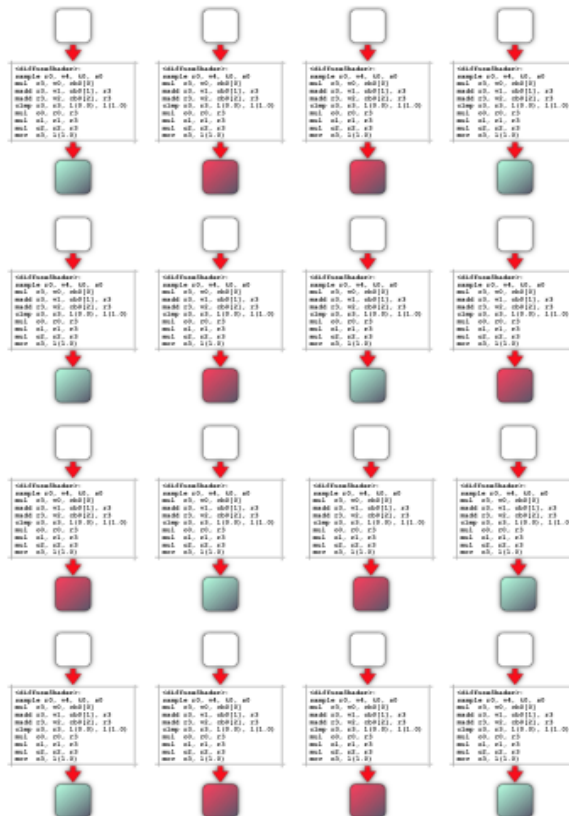
➤ Exploit the abundant parallelism



Graphics Processing Units

Since all invocations of kernel are independent, run on many cores

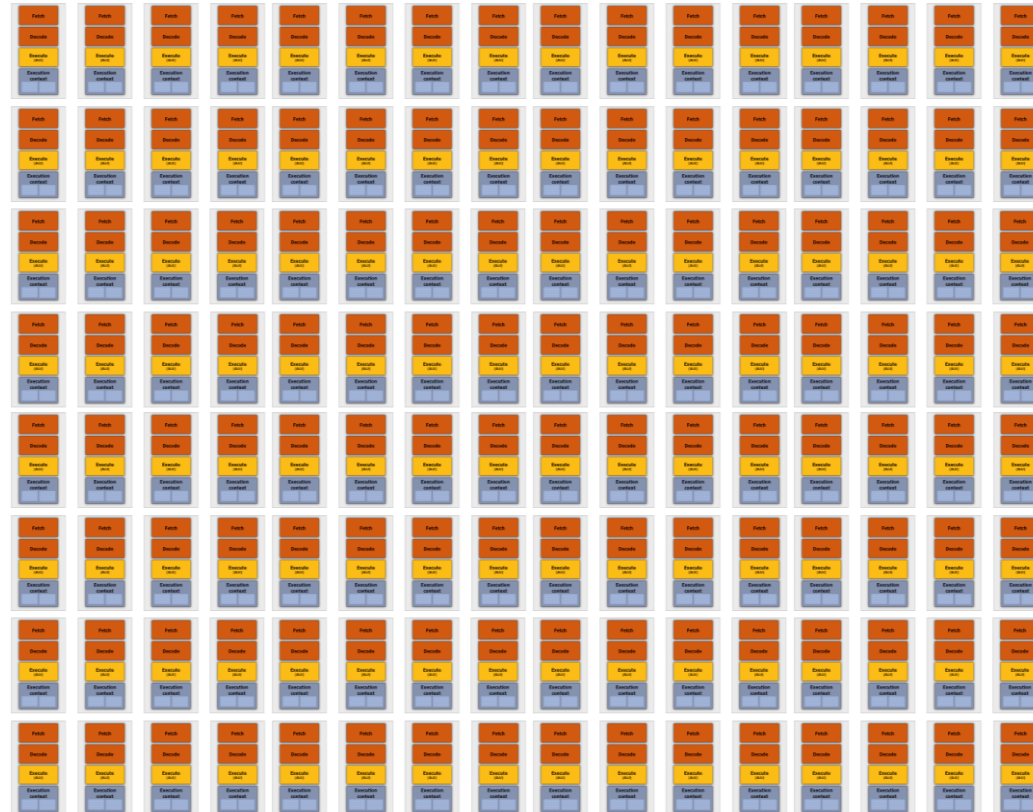
- Exploit the abundant parallelism



Graphics Processing Units

Since all invocations of kernel are independent, run on many cores

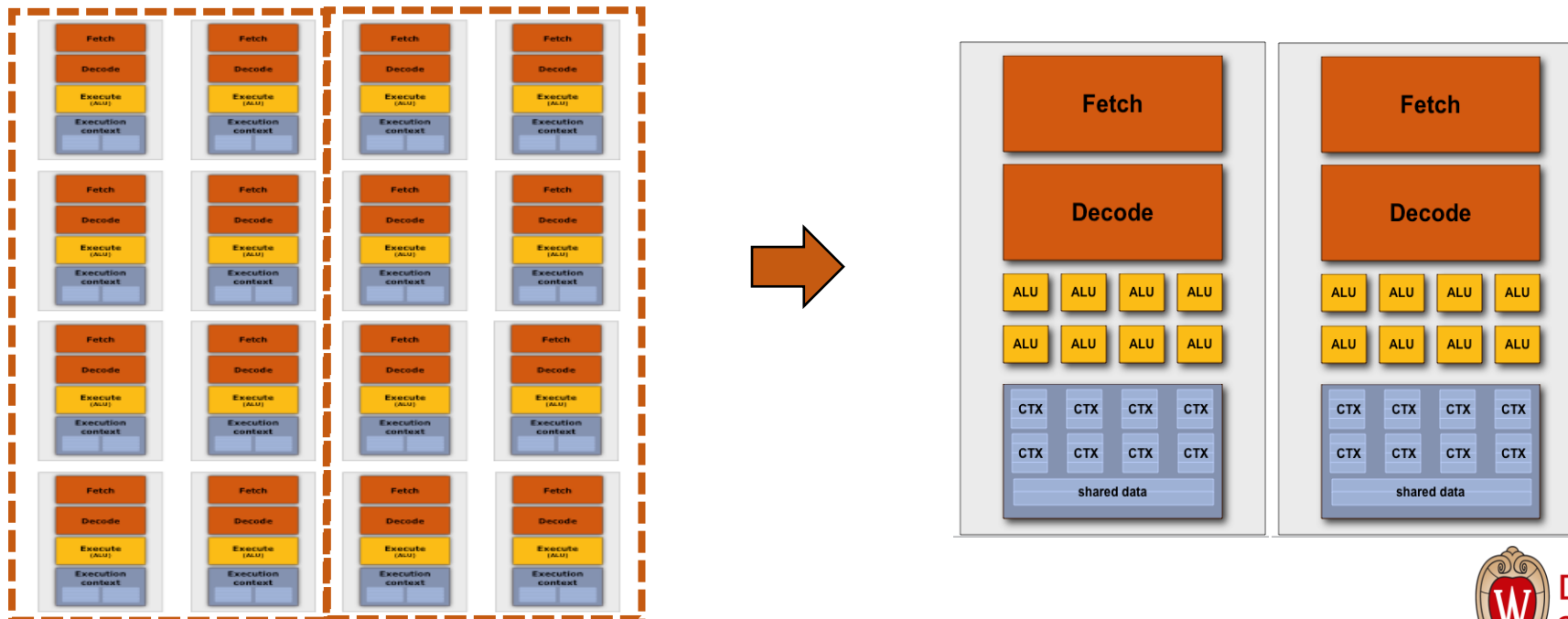
➤ Exploit the abundant parallelism



Graphics Processing Units

How do we feed all these cores?

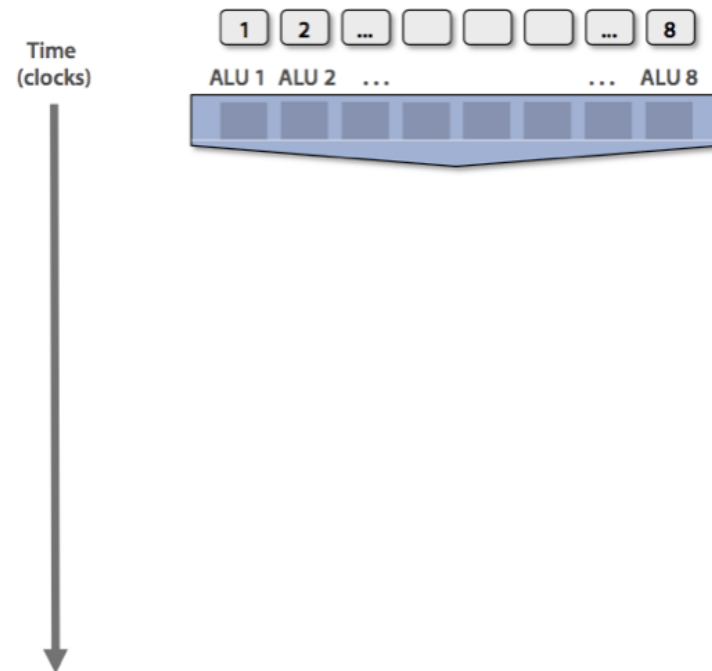
- Share cost of fetch+decode among chunks of threads
- All threads in a chunk execute in lockstep; usually simple in-order pipeline
 - Like SIMD instructions, except multiple thread contexts



Graphics Processing Units

What about conditional branches?

- All basic blocks (i.e., if, else if, else) are executed
 - “If” threads do nothing during “else” block, and vice versa
 - Execution units underutilized if lots of *branch divergence*

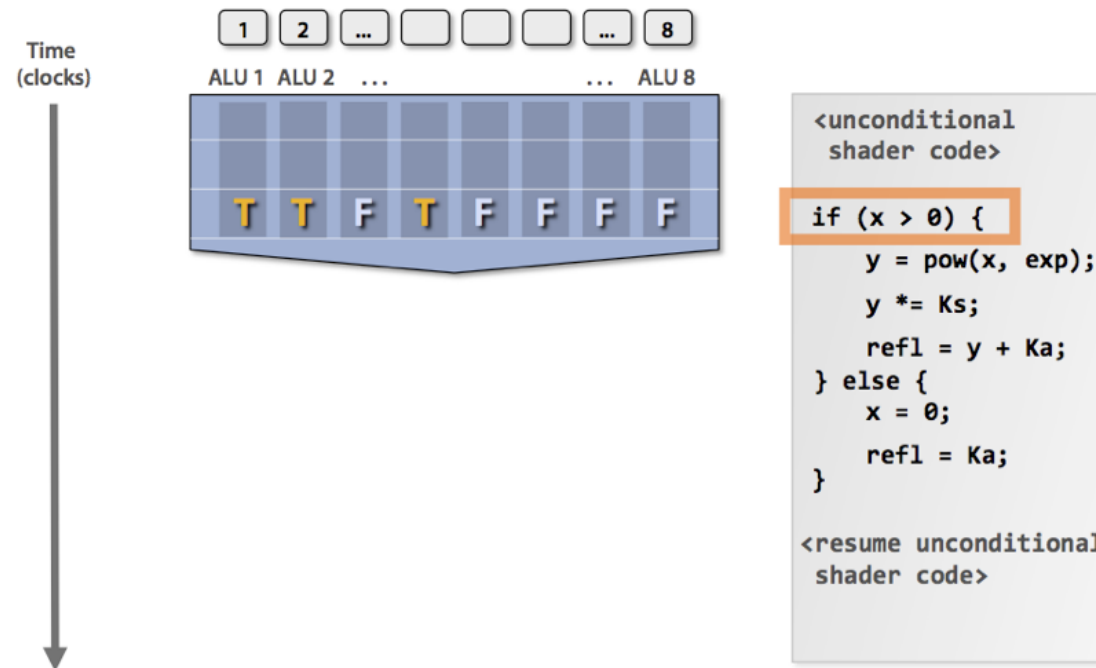


```
<unconditional  
shader code>  
  
if (x > 0) {  
    y = pow(x, exp);  
    y *= Ks;  
    refl = y + Ka;  
} else {  
    x = 0;  
    refl = Ka;  
}  
  
<resume unconditional  
shader code>
```

Graphics Processing Units

What about conditional branches?

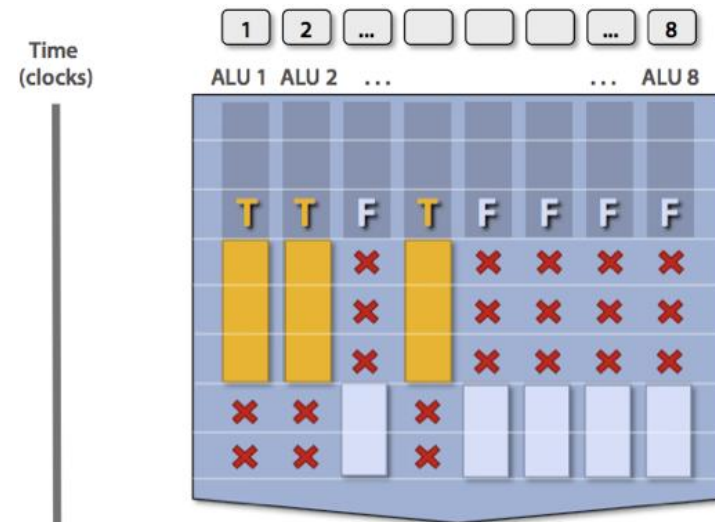
- All basic blocks (i.e., if, else if, else) are executed
 - “If” threads do nothing during “else” block, and vice versa
 - Execution units underutilized if lots of *branch divergence*



Graphics Processing Units

What about conditional branches?

- All basic blocks (i.e., if, else if, else) are executed
 - “If” threads do nothing during “else” block, and vice versa
 - Execution units underutilized if lots of *branch divergence*



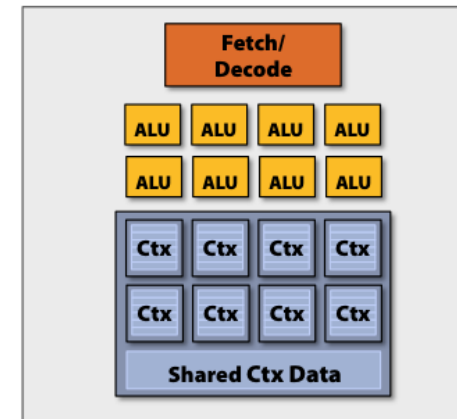
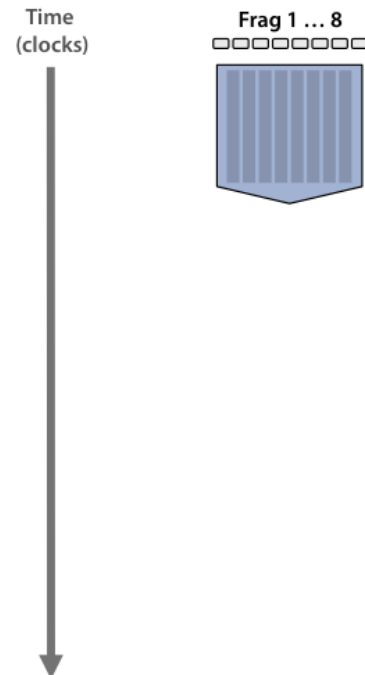
e.g., worst case:
1/8 performance
(different condition per thread)

```
<unconditional  
shader code>  
  
if (x > 0) {  
    y = pow(x, exp);  
    y *= Ks;  
    refl = y + Ka;  
} else {  
    x = 0;  
    refl = Ka;  
}  
  
<resume unconditional  
shader code>
```

Graphics Processing Units

What about memory stalls?

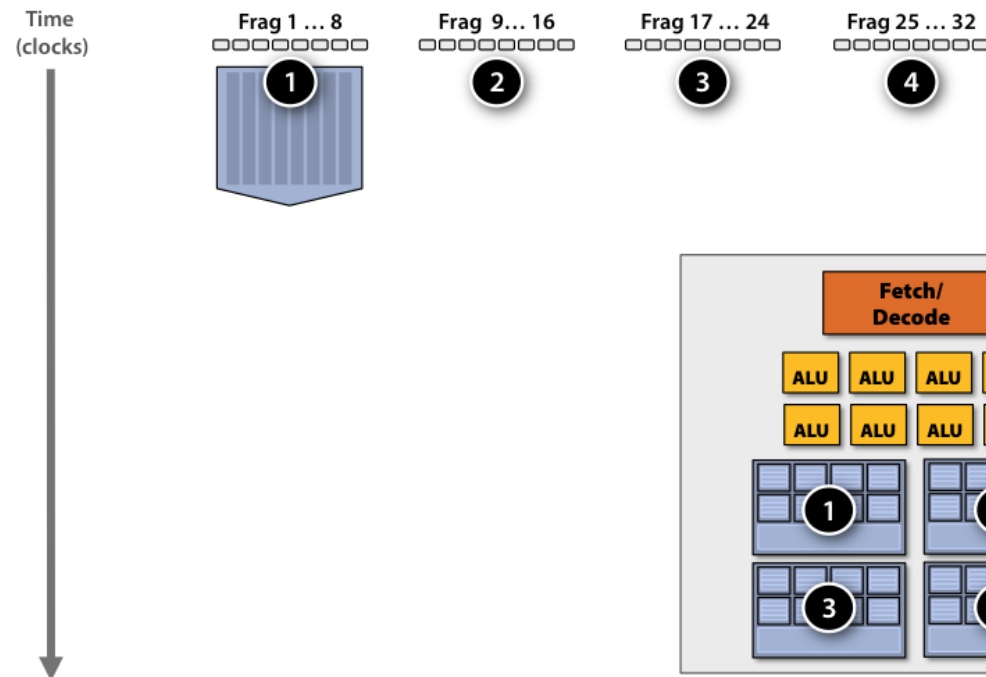
- Hide stall latencies by executing many more software threads than available hardware cores
 - Interleave execution of chunks of threads via fine-grained context-switching



Graphics Processing Units

What about memory stalls?

- Hide stall latencies by executing many more software threads than available hardware cores
 - Interleave execution of chunks of threads via fine-grained context-switching

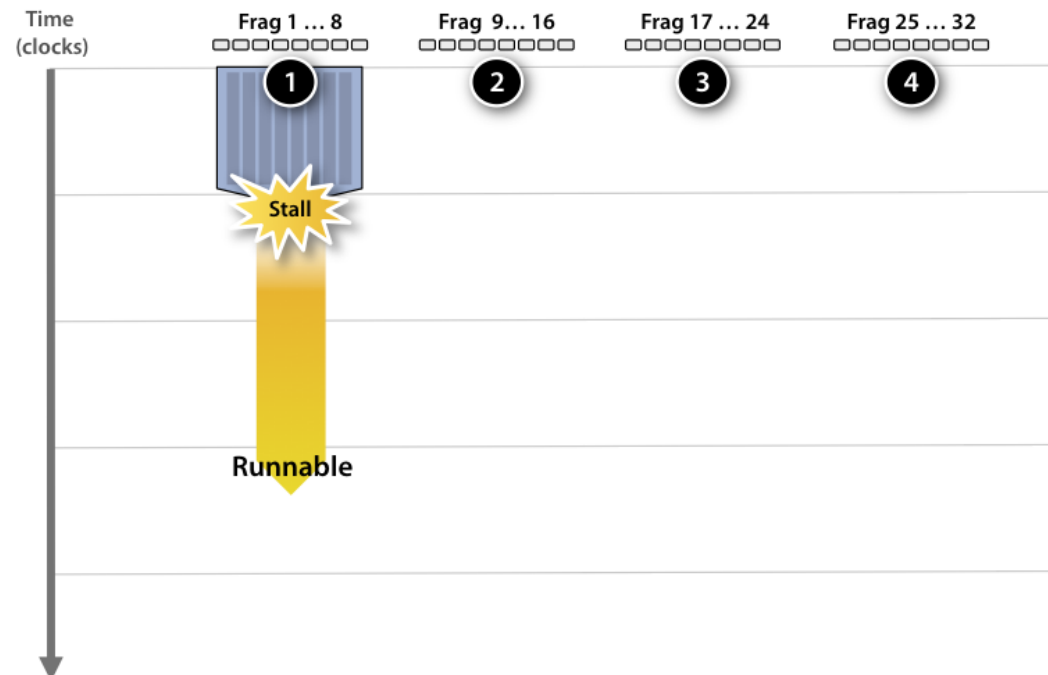


need to store contexts of multiple chunks of threads

Graphics Processing Units

What about memory stalls?

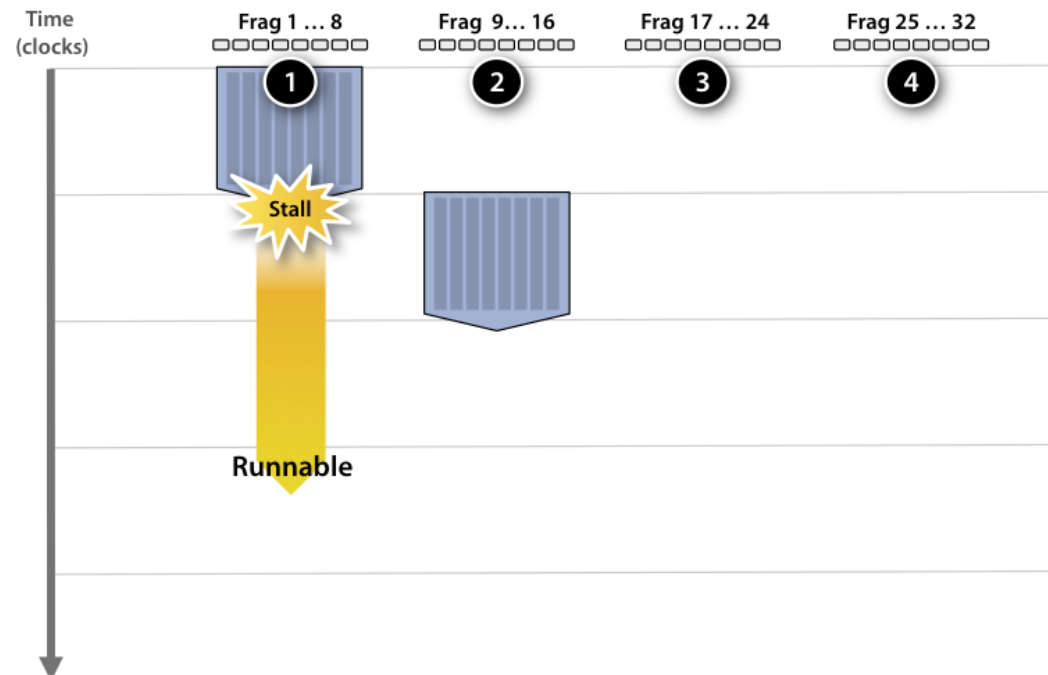
- Hide stall latencies by executing many more software threads than available hardware cores
 - Interleave execution of chunks of threads via fine-grained context-switching



Graphics Processing Units

What about memory stalls?

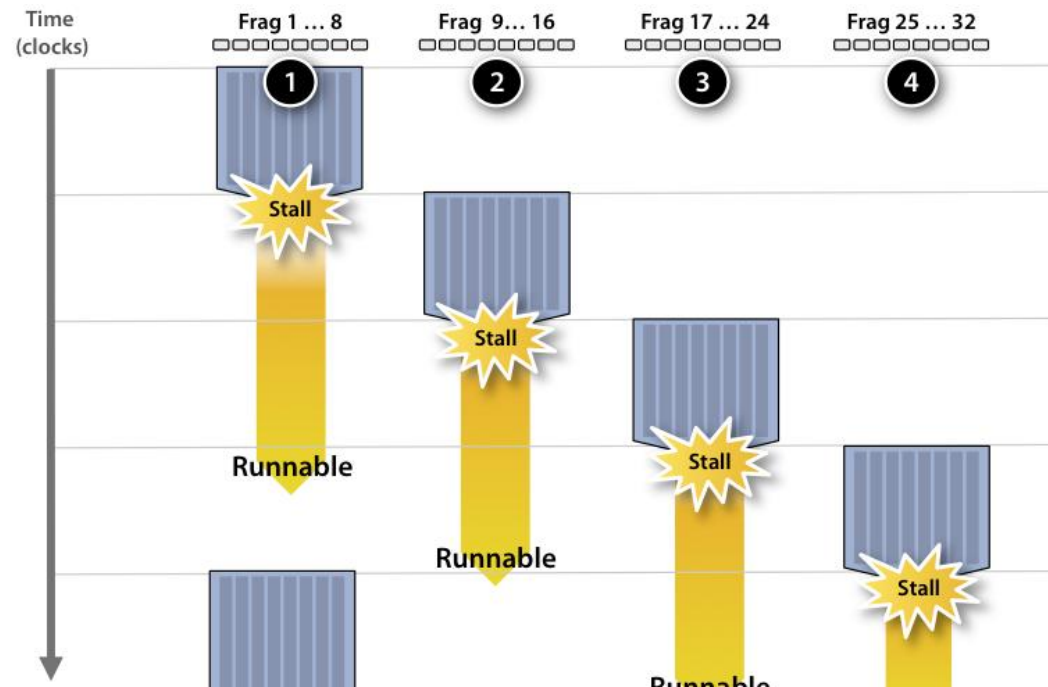
- Hide stall latencies by executing many more software threads than available hardware cores
 - Interleave execution of chunks of threads via fine-grained context-switching



Graphics Processing Units

What about memory stalls?

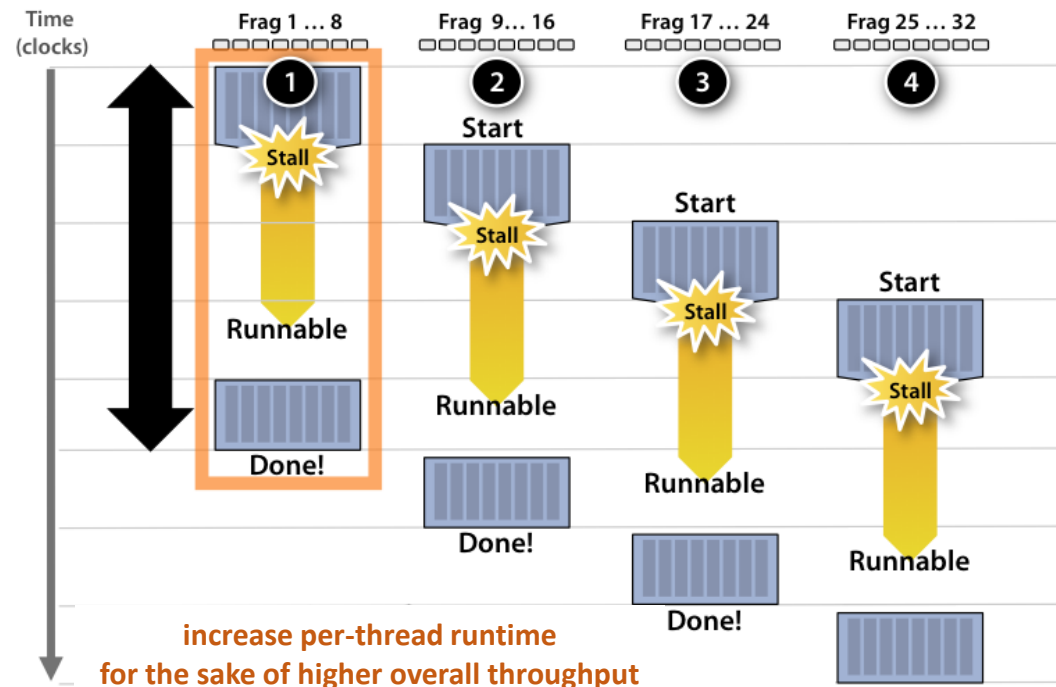
- Hide stall latencies by executing many more software threads than available hardware cores
 - Interleave execution of chunks of threads via fine-grained context-switching



Graphics Processing Units

What about memory stalls?

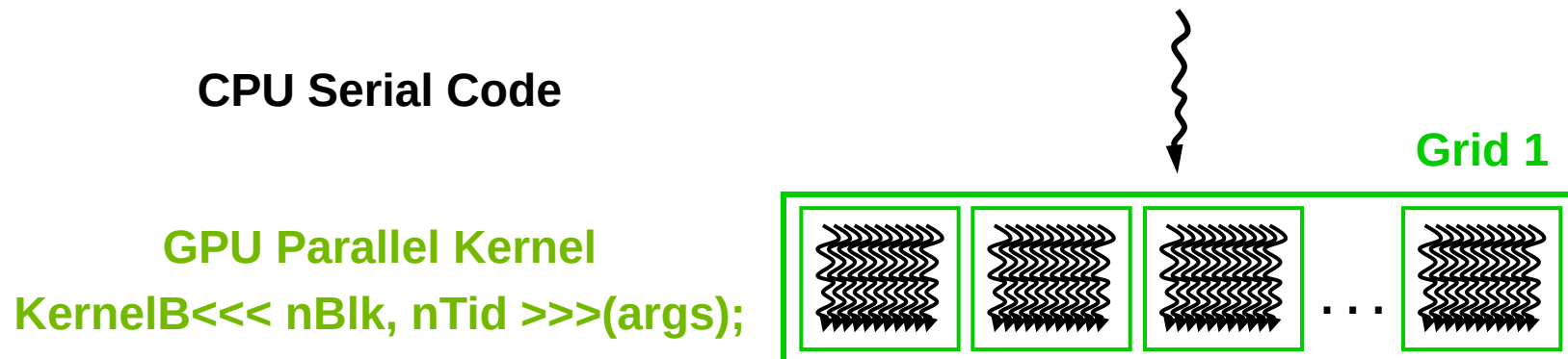
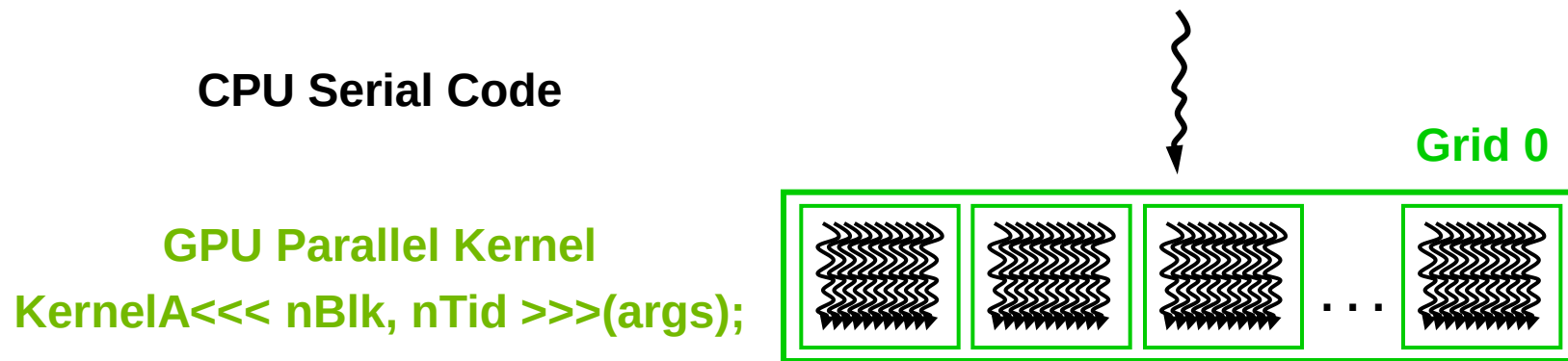
- Hide stall latencies by executing many more software threads than available hardware cores
 - Interleave execution of chunks of threads via fine-grained context-switching



GPU Threading

CUDA integrated CPU+GPU program

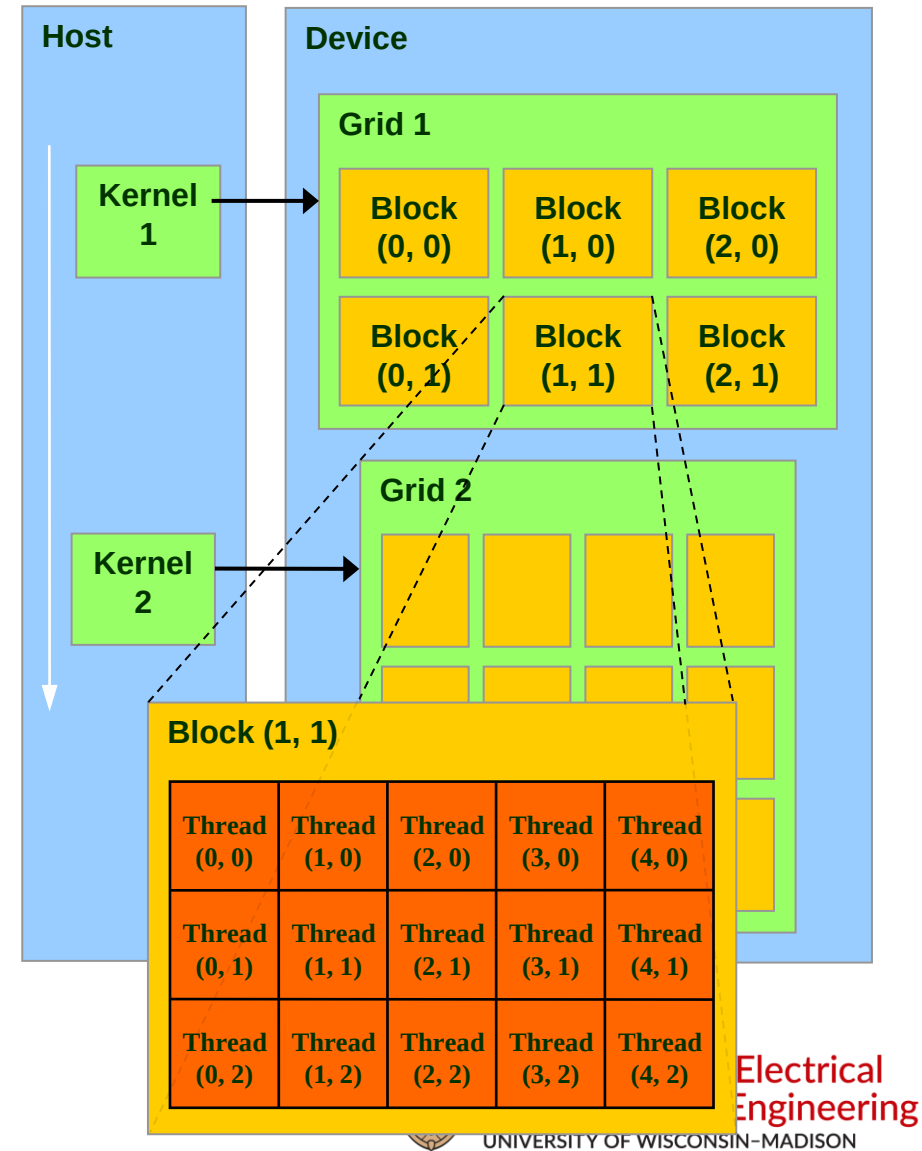
- Serial C code executes on CPU
- Parallel kernel C code executes on GPU thread blocks



GPU Threading

A kernel is executed as a *grid of thread blocks*

- All threads share memory space



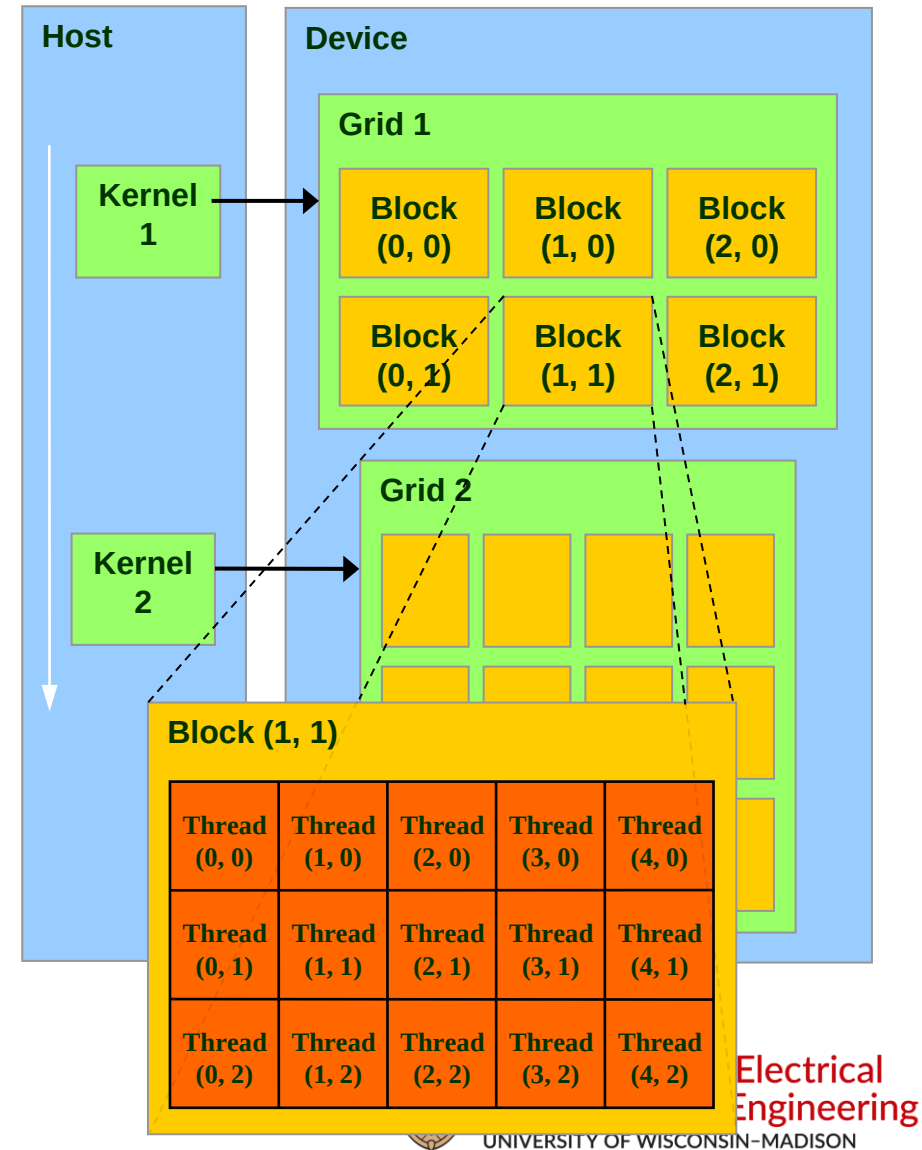
GPU Threading

A kernel is executed as a *grid of thread blocks*

- All threads share memory space

A thread block is a batch of threads that can cooperate with each other:

- Synchronizing their execution, for hazard-free shared memory accesses
- Data transfer through a low-latency shared memory
- Two threads from two different blocks cannot cooperate

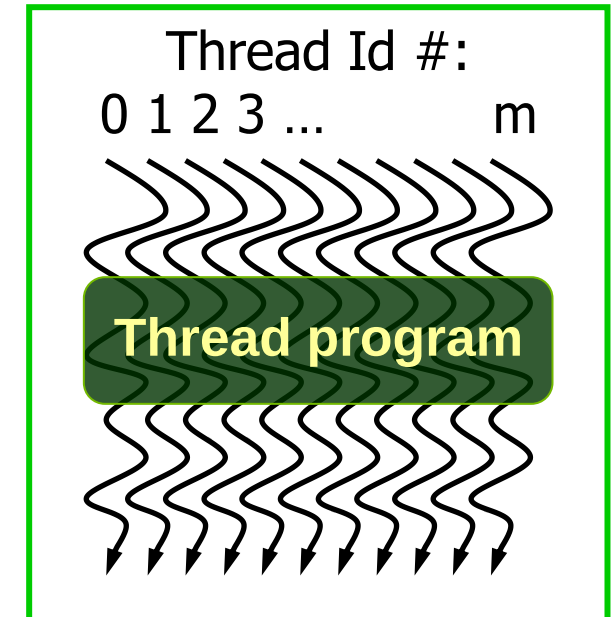


GPU Threading

Thread blocks:

- All threads in a thread block execute on same machine
- Programmer declares block size (e.g., 1-100s of concurrent threads), block shape (e.g., 1D, 2D, 3D), block dimensions (in threads)
- Each thread has thread ID within block; uses this thread ID to select which work to do and which data addresses to access

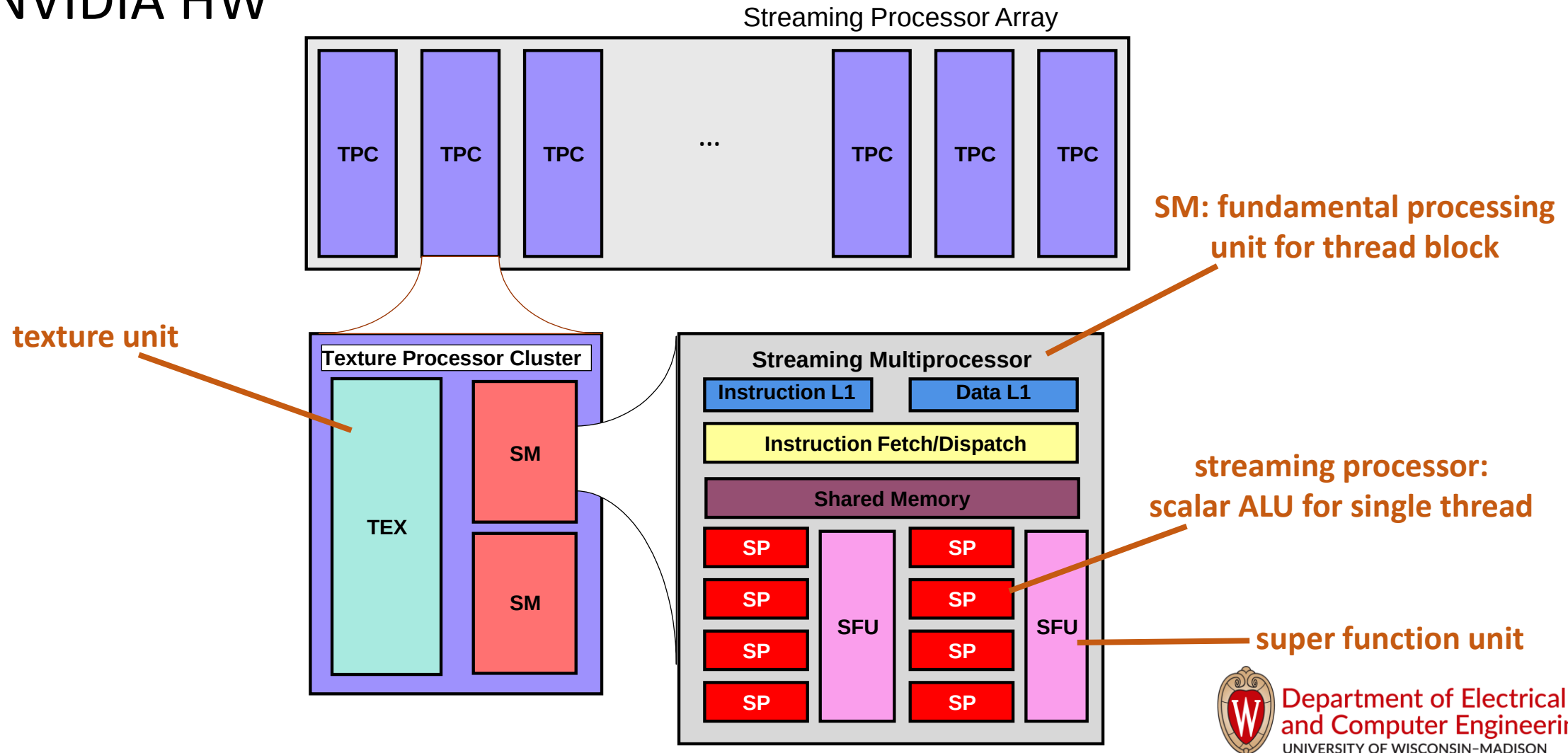
CUDA Thread Block



source: John Nickolls, NVIDIA

GPU Architecture

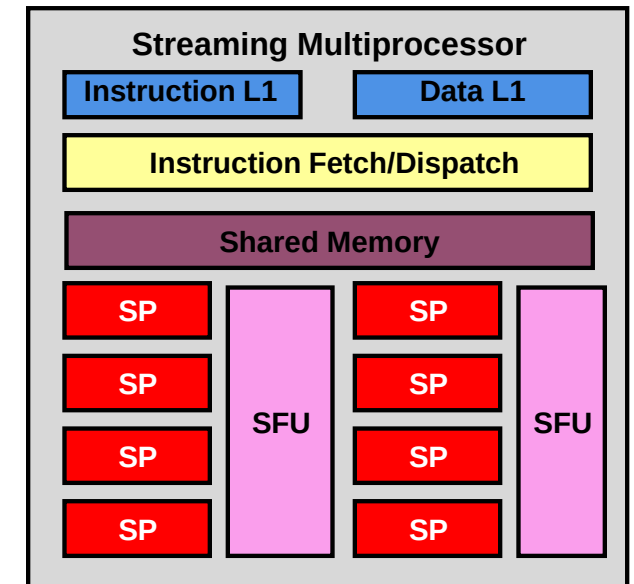
e.g., NVIDIA HW



GPU Architecture

Streaming Multiprocessor (SM):

- Multithreaded instruction dispatch
- Supports many (100s-1000s) active thread contexts
- Shared instruction fetch per chunk (*warp*) of threads
- e.g., Typically:
 - 32 threads per warp
 - 16-128 KB shared memory



GPU Architecture – Thread Blocks

Streaming processor array launches grid

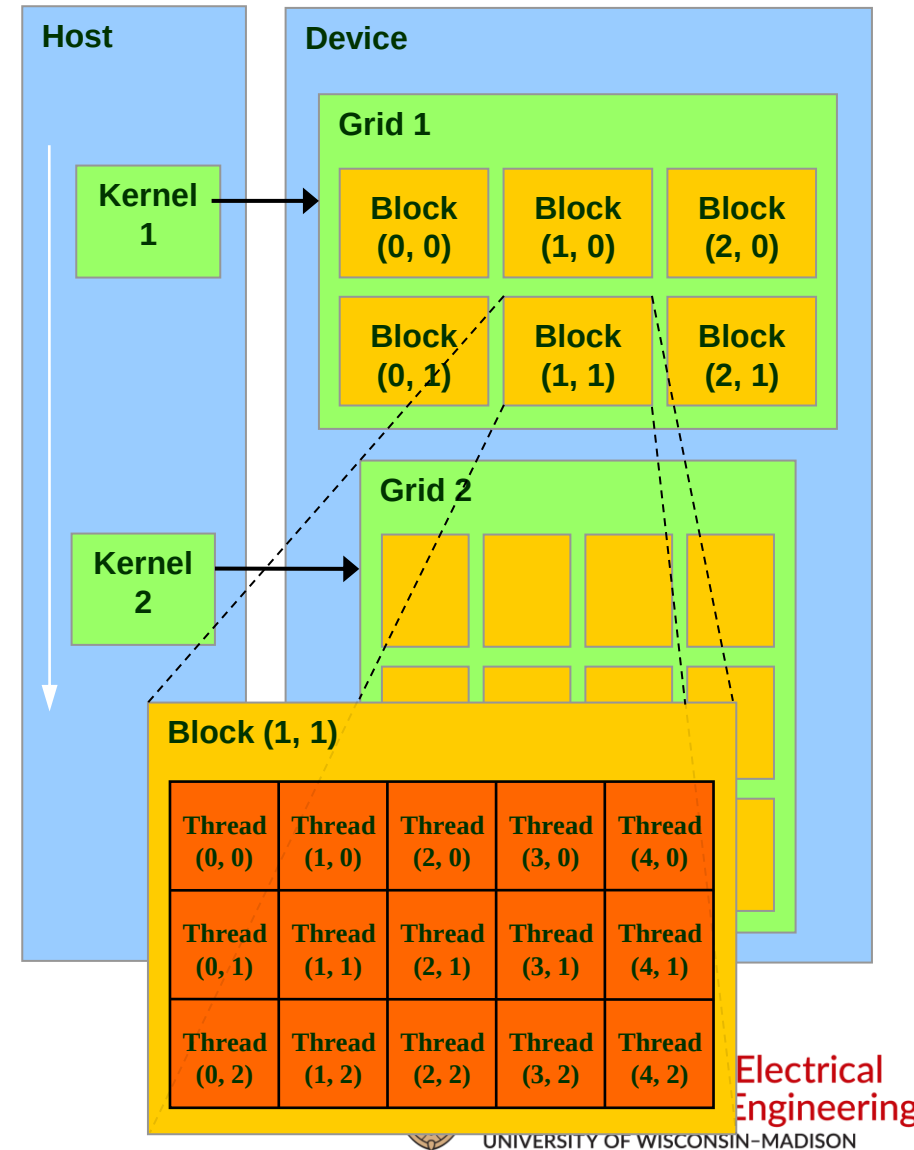
- Thread blocks are serially distributed to all SMs

Each SM launches warps of threads

- SM schedules and executes (time-multiplexes) warps that are ready to run

As warps and thread blocks complete, resources are freed

- Streaming processor array can distribute more thread blocks



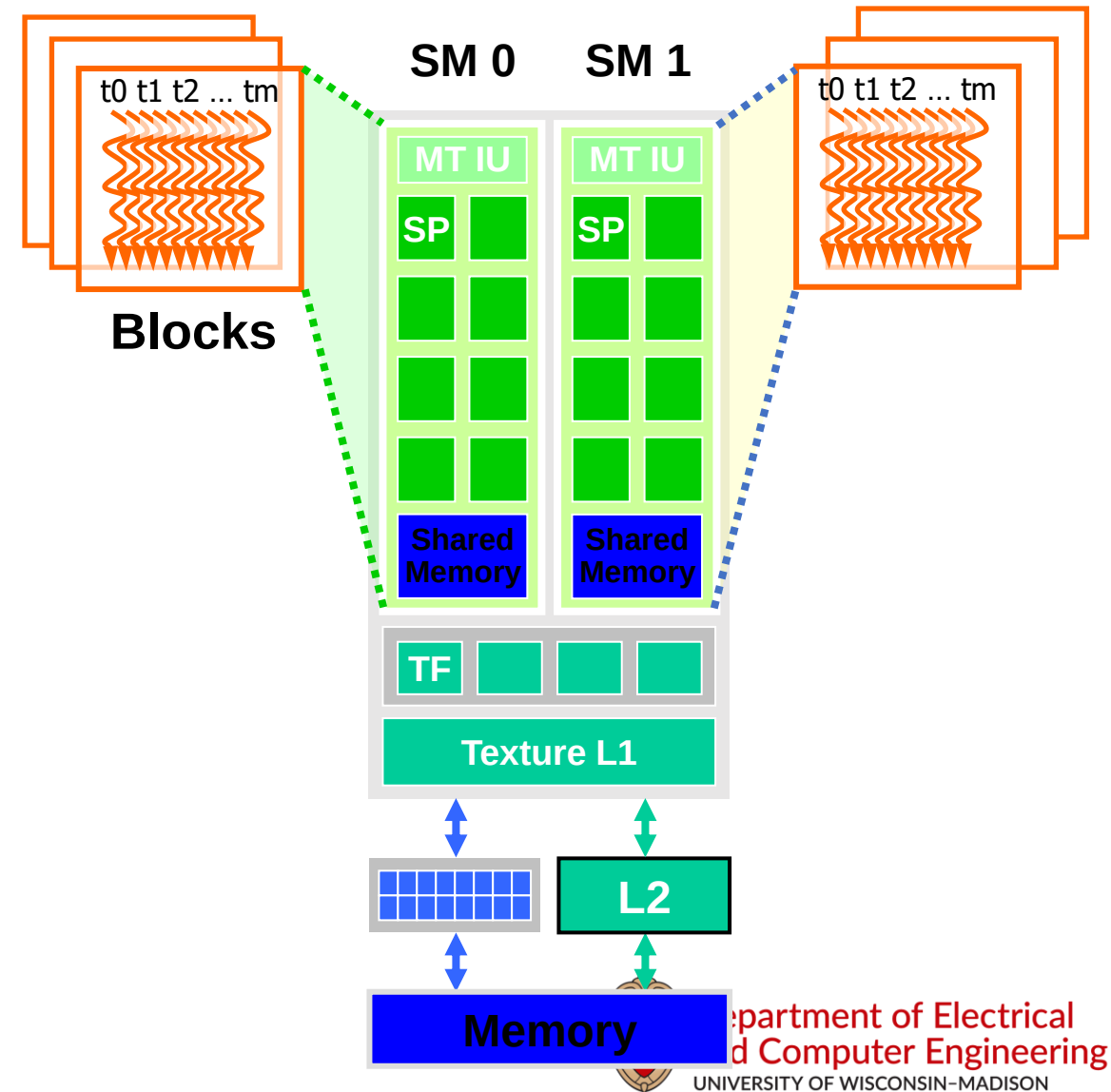
GPU Architecture – Thread Blocks

Threads are assigned to SMs in block granularity

- 10s of blocks per SM as resources allow
- 100s-1000s of threads per SM

Threads run concurrently

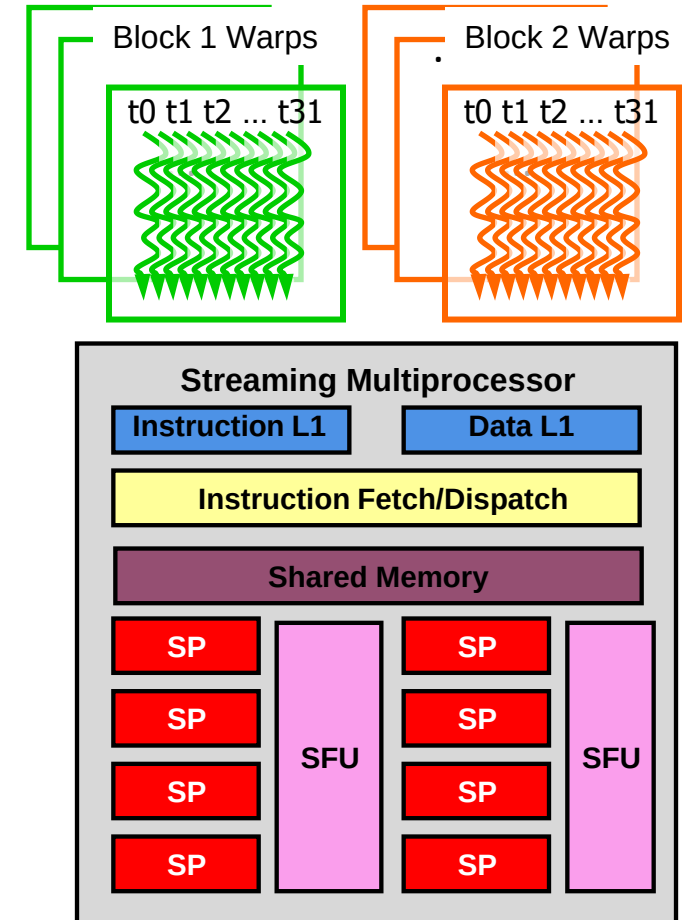
- SM assigns and maintains thread IDs
- SM manages and schedules thread execution



GPU Architecture – Warps

Each thread block is divided into *warps*

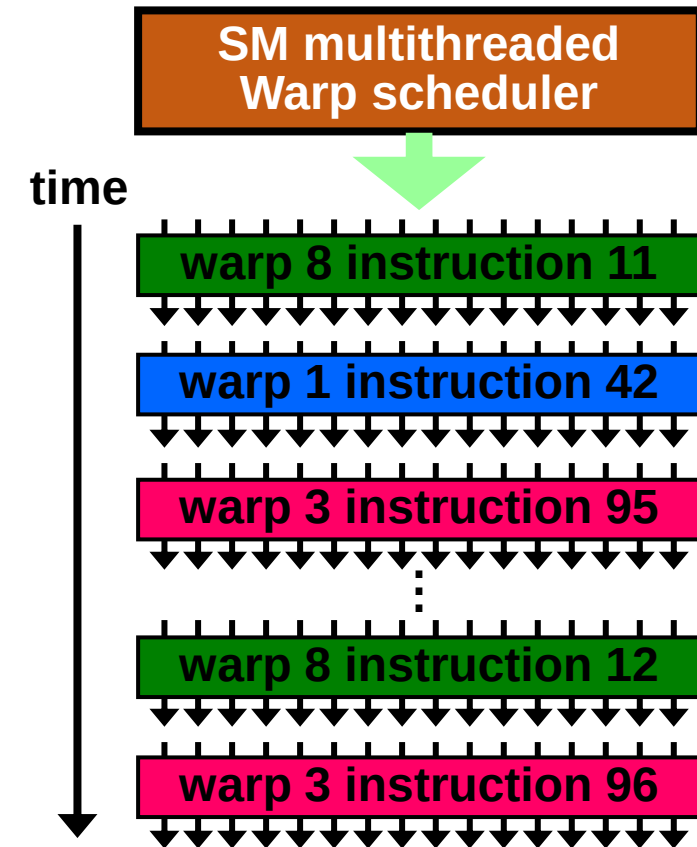
- HW implementation decision (not defined by programming model)
- Warp is a set of threads that always execute in lockstep
- Warps are the units for scheduling inside SM
 - At any point in time, only one warp is selected for instruction fetch



GPU Architecture – Warps

SM hardware implements zero-overhead warp scheduling

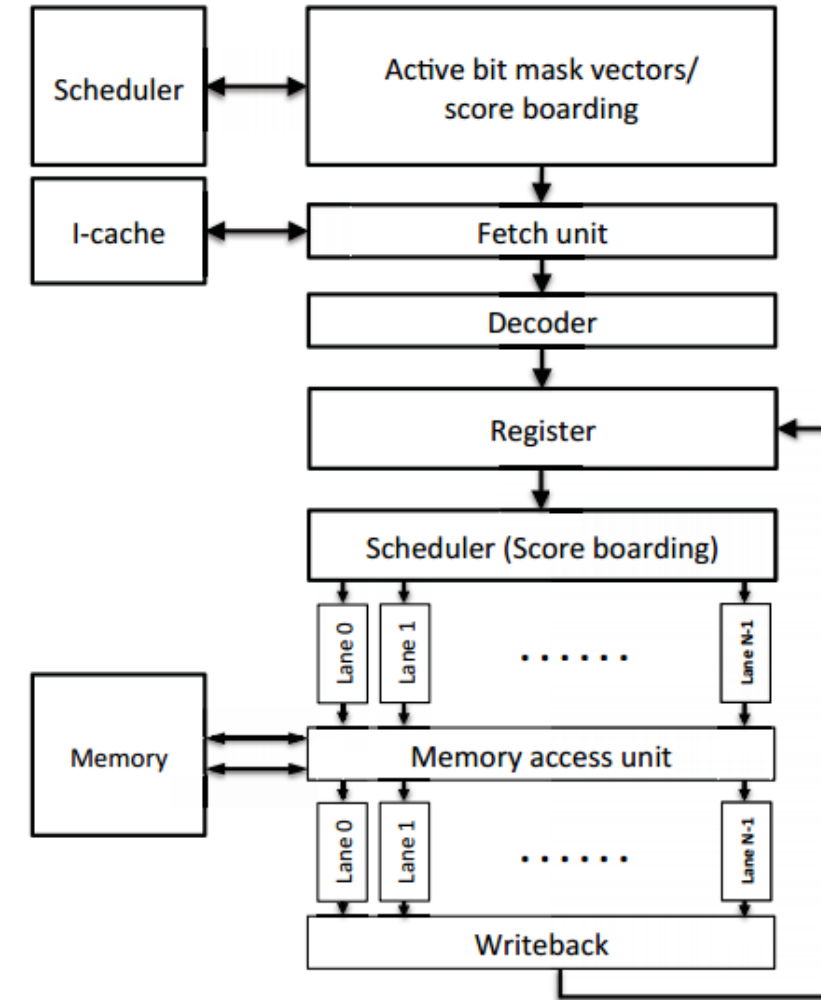
- Warps whose next instruction has its operands ready are eligible for execution
- Eligible warps are selected for execution via prioritized scheduling policy
- Interleave warp execution to hide latencies (e.g., memory)



GPU Architecture – Warps

Fetch one *warp instruction* per cycle

- From instruction L1 cache
- Into any instruction buffer slot



GPU Architecture – Warps

Fetch one *warp instruction* per cycle

- From instruction L1 cache
- Into any instruction buffer slot

Issue one ready-to-go warp instruction per cycle

- From any warp
- Issued to execution lanes
- Operand scoreboarding used to prevent hazards
- Issue selection based on round-robin/age of warp

